Automating layer management



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Release management

There are multiple tasks that OE/bitbake based projects let you do on your own to ensure build reproducibility:

- Code distribution and project setup
- Release tagging

A separate tool is needed for that, usual solutions are:

- git submodules + setup script. Great example in YOE: https://github.com/YoeDistro/yoe-distro
- repo and `templateconf` or setup script: pretty popular in the past, before dedicated tools emerged
- kas: probably the most popular tool
- bitbake-setup: the newest and official tool



kas

- Specific tool developed by Siemens for OpenEmbedded: https://github.com/siemens/kas
- Will fetch layers and build the image in a single command
- Uses a single JSON or YAML configuration file part of the custom layer
- Can generate and run inside a Docker container
- Can setup `local.conf` and `bblayers.conf`



kas configuration

```
header:
  version: 8
machine: mymachine
distro: mydistro
target:
  - myimage

repos:
  meta-custom:

  bitbake:
    url: "https://git.openembedded.org/bitbake"
    # tag 2.0
    commit: c212b0f3b542efa19f15782421196b7f4b64b0b9
    layers:
      .: excluded

  openembedded-core:
    url: "https://git.openembedded.org/openembedded-core"
    branch: wrynose
    layers:
      meta:
```



kas configuration

```
meta-freescale:
  url: "https://github.com/Freescale/meta-freescale"
  branch: wrynose

meta-openembedded:
 url: https://git.openembedded.org/meta-openembedded
 branch: wrynose
 layers:
   meta-oe:
   meta-python:
   meta-networking:
```

- Then a single command will build all the listed targets for the machine:

```
$ kas build meta-custom/mymachine.yaml
```

- Or, alternatively, invoke `bitbake` commands:

```
$ kas shell /path/to/kas-project.yml -c 'bitbake dosfsutils-native'
```



bitbake-setup

- The official tool, part of bitbake git: https://git.openembedded.org/bitbake/tree/bin/bitbake-setup
- Will fetch layers and setup the build configuration
- Does not try to mask core tools, users still need to launch builds with bitbake
- Uses a JSON configuration file



bitbake-setup configuration

```
{
    "description": "My bitbake setup file.",
    "sources": {
        "bitbake": {
            "git-remote": {
                "uri": "https://git.openembedded.org/bitbake",
                "rev": "2.18"
            }
        },
        "openembedded-core": {
            "git-remote": {
                "uri": "https://git.openembedded.org/openembedded-core",
                "rev": "wrynose"
            }
        },
        "meta-openembedded": {
            "git-remote": {
                "uri": "https://git.openembedded.org/meta-openembedded",
                "rev": "wrynose"
            }
        },
        "meta-freescale": {
            "git-remote": {
                "uri": "https://github.com/Freescale/meta-freescale",
                "rev": "wrynose"
            }
        }
    },
```



bitbake-setup configuration

```
    "bitbake-setup": {
        "configurations": [{
            "name": "myconfig",
            "description": "My own configuration",
            "bb-layers": ["openembedded-core/meta", "meta-openembedded/meta-oe",
                          "meta-openembedded/meta-python", "meta-freescale"],
            "oe-fragments": ["machine/mymachine", "distro/mydistro"]
        }]
    },
    "version": "1.0"
}
```

- Then a single command will fetch all layers and setup your configuration:

```
$ bitbake-setup init mysetup.conf.json
```

- This will also create an environment file that you can source:

```
$ . ./mysetup-myconfig/build/init-build-env
$ bitbake myimage
```

OTA updates



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Runtime Package Management



Runtime Package Management

- BitBake always builds packages selected in IMAGE_INSTALL.
- The binary packages are used to generate the root filesystem.
- It is also possible to update the system at runtime using these packages, for many use cases:
  - In-field security updates.
  - System updates over the wire.
  - System, packages or configuration customization at runtime.
  - Remote debugging.
- Using the Runtime Package Management is an optional feature.
- Can be useful for local development, but rarely used in production.
- See the extra slides for details on how to set it up.



A/B updates



A/B updates: principle

- Mechanism for Over-The-Air (OTA) *image* updates
- Key idea: avoid one update bricking the device
  - Even if tested, one update might prevent the system from booting
  - Enable the device to recover from a bad update automatically
- The key idea is to maintain at least two copies, in separate partitions (slots) of your storage device
  - minimum is 2 rootFS
  - usually also includes the kernel
  - can involve the bootloader, depending on the ROM code



A/B updates: Yocto support

- Three main open-source solutions:
  - SWUpdate:
    - DIY approach, very flexible
    - Yocto layer: meta-swupdate
  - Mender:
    - Fully-integrated solution, requires `systemd`
    - Yocto layer: meta-mender
  - RAUC:
    - Good middleground
    - Yocto layer: meta-rauc
- See our Security training for in-depth details on A/B updates.

Yocto Project Resources



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Yocto Project documentation

- https://docs.yoctoproject.org/
- Wiki: https://wiki.yoctoproject.org/wiki/Main_Page
- https://layers.openembedded.org/



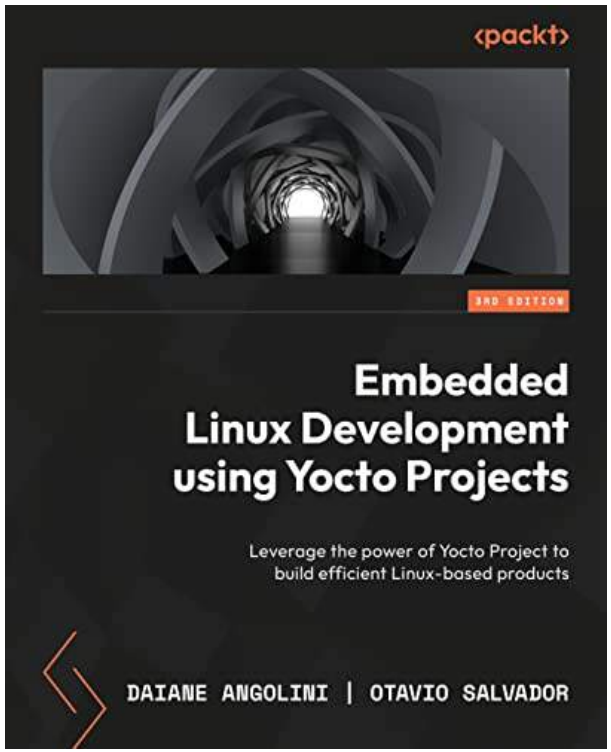
Contributing

- Friendly community to newcomers of the project
- Mailing list based upstreaming workflow:
  https://docs.yoctoproject.org/contributor-guide
- Ask questions and help people on IRC:
  https://web.libera.chat/?channels=#yocto
- Try to solve newcomer bugs reported on the official Bugzilla:
  https://wiki.yoctoproject.org/wiki/Newcomers#Newcomer_Bugs
- The Wiki contains a lot of useful information:
  https://wiki.yoctoproject.org/wiki/Main_Page

Useful Reading

Embedded Linux Development Using Yocto Project - Third Edition, April 2023

- https://www.packtpub.com/en-us/product/embedded-linux-development-using-yocto-project-9781804615065
- By Daiane Angolini and Otavio Salvador
- From basic to advanced usage, helps writing better, more flexible recipes. A good reference to jumpstart your Yocto Project development.



Last slides

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!





Last slide

Thank you!

And may the Source be with you



Rights to copy

© Copyright 2004-2026, Bootlin
License: Creative Commons Attribution - Share Alike 3.0
https://creativecommons.org/licenses/by-sa/3.0/legalcode
You are free:

- to copy, distribute, display, and perform the work
- to make derivative works
- to make commercial use of the work

Under the following conditions:

- **Attribution**. You must give the original author credit.
- **Share Alike**. If you alter, transform, or build upon this work, you may distribute the resulting work only under a license identical to this one.
- For any reuse or distribution, you must make clear to others the license terms of this work.
- Any of these conditions can be waived if you get permission from the copyright holder.

Your fair use and other rights are in no way affected by the above.

Document sources: https://github.com/bootlin/training-materials/

Extra slides

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!





Quilt



Overview

- Quilt is a utility to manage patches which can be used without having a clean source tree.
- It can be used to create patches for recipes already available in the build system.
- Be careful when using this workflow: the modifications won't persist across builds!



Using Quilt

1. Find the recipe working directory in `$BUILDDIR/tmp/work/`.
2. Create a new `Quilt` patch: `$ quilt new topic.patch`
3. Add files to this patch: `$ quilt add file0.c file1.c`
4. Make the modifications by editing the files.
5. Test the modifications: `$ bitbake -c compile -f recipe`
6. Generate the patch file: `$ quilt refresh`
7. Move the generated patch into the recipe's directory.



Runtime Package Management (IPK Setup)



Requirements

- First of all, you need a server to serve the packages to a private subnet or over the Internet. Packages are typically served over `https` or `http`.
- Specific tools are also required on the target, and must be shipped on the product. They should be included into the images generated by the build system.
- These tools will be specific to the package type used.
  - This is similar to Linux distributions: Debian is using `.deb` related tools (dpkg, apt…) while Fedora uses `.rpm` related ones (rpm, dnf).



Build configuration 1/2

- The PACKAGE_CLASSES variable controls which package format to use. More than one can be used.
- Valid values are `package_rpm`, `package_deb`, `package_ipk`.
- By default Poky uses the RPM format, while OpenEmbedded-Core uses the IPK one.
- Example:
  - `PACKAGE_CLASSES = "package_ipk"`
  - `PACKAGE_CLASSES = "package_rpm package_deb"`



Build configuration 2/2

To install the required tools on the target, there are two possible solutions:

- By adding `package-management` to the images features.
  - The required tool will be installed on the target.
  - The package database corresponding to the build will be installed as well.
- Or by manually adding the required tools in IMAGE_INSTALL. For example, to use the IPK format we need `opkg`.



Build considerations

- The Runtime Package Management uses package databases to store information about available packages and their version.
- Whenever a build generates a new package or modifies an existing one, the package database must be updated.
- `$ bitbake package-index`
- Be careful: BitBake does not properly schedule the `package-index` target. You must use this target alone to have a consistent package database.
  - `$ bitbake ninvaders package-index` won't necessarily generate an updated package database.



Package server configuration: Apache2 example setup

Apache2 HTTP setup for IPK packages. This should go in `/etc/apache2/sites-enabled/package-server.conf`.

```
<VirtualHost *:80>
    ServerName packages.example.net

    DocumentRoot /path/to/build/tmp/deploy/ipk
    <Directory /path/to/build/tmp/deploy/ipk>
        Options +Indexes
        Options Indexes FollowSymLinks
        Order allow,deny
        allow from all
        AllowOverride None
        Require all granted
    </Directory>
</VirtualHost>
```



Target configuration: configuring opkg

- The IPK runtime management software is `opkg`.
- It can be configured using configurations files ending in `.conf` in `/etc/opkg/`.
- This configuration helps `opkg` to find the package databases you want to use.
- For example, with our previously configured package server:

```
src/gz all http://packages.example.net/all
src/gz armv7a http://packages.example.net/armv7a
src/gz beaglebone http://packages.example.net/beaglebone
```

- This can be automatically generated by defining the PACKAGE_FEED_URIS, PACKAGE_FEED_BASE_PATHS and PACKAGE_FEED_ARCHS variables



opkg usage

- `opkg update`: fetch and update the package databases, from the remote package servers.
- `opkg list`: list available packages.
- `opkg upgrade`: upgrade all installed packages.
- `opkg upgrade <package>`: upgrade one package explicitly.
- `opkg install <package>`: install a specific package.



`opkg` upgrade over an unstable network

- To avoid upgrade issues when downloading packages from a remote package server using an unstable connection, you can first download the packages and then proceed with the upgrade.
- To do this we must use a cache, which can be defined in the `opkg` configuration with: `option cache /tmp/opkg-cache`.

```
# opkg update
# opkg --download-only upgrade
# opkg upgrade
```